

Python Django Developer — Database & ORM Test

Total Marks: ____ | Time: ____ minutes | Questions: ____

Candidate name: _____ Date: _____

Instructions: This paper is about a calling app where users talk to counselors and pay from a wallet. First read the story and the two rules below. The first question asks you to design the database tables yourself. The later questions ask you to fetch data from those tables using a Django ORM queryset. You do not need to write views. If you cannot recall the exact name of a function, write your idea clearly in code form — a correct approach still earns marks.

The Story

We are building a phone-calling app. A **user** calls a **counselor** for advice. The user pays for the call from money kept in their **wallet**. Every counselor has a per-minute price. The app keeps a record of every call and every wallet change, and it also keeps a record of the money that has to be paid to each counselor.

Two special rules apply. Read them carefully, because the questions depend on them.

Rule 1 — The offer recharge (double money)

- Every **new** user starts with an offer active on their account.
- While the offer is active, if the user recharges their wallet, they get **double** the amount. For example, a recharge of ■100 adds ■200 to the wallet.
- A recharge done while the offer is active is called an **offer transaction**. A recharge done when the offer is not active is a **normal transaction**.

Rule 2 — The first-call payment to the counselor

- A user's **first call that is longer than 60 minutes** is treated as that user's **first call**. Shorter calls before it do not count as the first call.
- For this one first call, the counselor is paid a **fixed ■20**, whatever their normal per-minute price is.
- For every other call, the counselor is paid in the normal way (by their own price). Each user can have only one such first call.

Questions

QUESTION 1

10 marks

Read the story and the two rules above. Design the database tables needed to run this app, and write them as Django models.

Your models should be able to hold everything the story talks about: the users, the counselors, every call, every wallet recharge (both offer and normal), and the money that has to be paid to each counselor. For each table, choose sensible fields and field types, mark the primary keys, and join the tables using foreign keys where they are related. Also add any field you think is needed to support Rule 1 and Rule 2.

You may write the models in plain code. Exact import lines are not required — clear field names, correct types and correct relationships are what matter.

[More questions to be added. From Question 2 onward the candidate will use the tables they designed above and answer each question with a single Django ORM queryset — for example the total offer money given this month, or the counselors who are still waiting to be paid.]